



Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



“Amazon Web Services (AWS): on-demand delivery of IT resources over the Internet.”

Amazon Web Services (AWS) is a complete cloud computing platform that lets you rent virtual computers, storage, and databases over the internet on a pay-as-you-go model. Amazon Web Services (AWS) core services are foundational cloud infrastructure tools—covering compute, storage, databases, networking, and security—that applications require to run and scale.

AWS consists of many cloud services that you can use in combinations tailored to your organizational needs. To access the services, you can use the AWS Management Console (a simple intuitive user interface), the Command Line Interface (CLI), or Software Development Kits (SDKs).

AWS provides building blocks that you can assemble quickly to support virtually any workload. With AWS, you'll find a complete set of highly available services that are designed to work together to build sophisticated scalable applications. Here are few services you must learn to perform most of the common task on AWS.

AWS is a real cloud platform. Creating resources can incur charges. Always check the pricing/free-tier eligibility of your account and region, and delete resources after the lab. Follow a flow as rule:

CREATE → CONFIGURE → TEST → OBSERVE → DELETE

While AWS technically bills based on exact usage (often down to the millisecond or gigabyte), several core services are famous for costing \$0 to \$1 for small-scale, hobby, or learning projects.

Core Compute Services

- **Amazon EC2 (Elastic Compute Cloud):** Provides resizable virtual servers in the cloud so you can run applications without buying physical hardware.
- **AWS Lambda:** Runs your code in a serverless environment in response to events, meaning you do not manage any underlying servers.

Core Storage Services

- **Amazon S3 (Simple Storage Service):** Offers scalable object storage to store and retrieve any amount of data, like files, images, and backups, from anywhere on the web.
- **Amazon EBS (Elastic Block Store):** Attaches persistent block storage (virtual hard drives) to your EC2 instances.

Core Database Services

- **Amazon RDS (Relational Database Service):** Manages relational SQL databases like MySQL, PostgreSQL, and Oracle easily.
- **Amazon DynamoDB:** Delivers a fast, predictable, and fully managed NoSQL database for high-performance applications.

Core Networking & Security Services

- **Amazon VPC (Virtual Private Cloud):** Sets up a logically isolated, private virtual network for your cloud resources.
- **AWS IAM (Identity and Access Management):** Controls user access and securely manages permissions to your AWS services.
- **Amazon CloudFront:** Speeds up the delivery of static and dynamic web content using a global content delivery network (CDN).





Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



Amazon EC2

Amazon EC2 (Elastic Compute Cloud) is a web service provided by AWS that gives you secure, resizable virtual servers in the cloud. Instead of buying physical hardware, you "rent" these virtual machines (called instances) on-demand to run applications, host websites, or process data, paying only for the compute capacity you use. Here is a step-by-step guide to creating and testing a basic Linux EC2 instance using the AWS Management Console.

Step 1: Launch an EC2 Instance

1. Log in to your AWS Management Console.
2. In the search bar at the top, type **EC2** and select **EC2 (Virtual servers in the cloud)**.
3. Click the orange **Launch instance** button.
4. **Name your instance:** Under *Name and tags*, enter a name (e.g., My-First-EC2).
5. **Choose an OS (AMI):** Under *Application and OS Images*, select **Amazon Linux** or **Ubuntu** (both have options that are Free Tier eligible).
6. **Select an Instance Type:** Keep the default **t2.micro** (or **t3.micro**, depending on your AWS region) to stay within the AWS Free Tier.
7. **Key pair:** Under *Key pair (login)*, click **Create new key pair**. Name it, keep the format as .pem, and download it. *Keep this file safe—you will need it if you want to connect via terminal later.*
8. **Network Settings:** Ensure **Allow SSH traffic from Anywhere (0.0.0.0/0)** is checked. Also check **Allow HTTP traffic from the internet** (this will help us test it as a web server).
9. **Launch:** Leave the storage settings as default and click **Launch instance**.

Step 2: Test the EC2 Instance (Connect & Run a Web Server)

The easiest way to test your new instance without configuring complex terminal tools is by using **EC2 Instance Connect**.

1. Wait a couple of minutes until your Instance State changes to **Running**.
2. Select your instance by clicking the checkbox next to it, then click the **Connect** button at the top.
3. Choose the **EC2 Instance Connect** tab and click **Connect**. A new browser tab will open showing a Linux command terminal.
4. To test that the instance can process commands and serve data, turn it into a quick web server by pasting the following commands into the browser terminal:

```
# Update the server packages
sudo yum update -y
# Install the Apache Web Server
sudo yum install -y httpd
# Start the web server
sudo systemctl start httpd
sudo systemctl enable httpd
```

Step 3: Verify It Works

1. Go back to your Amazon EC2 Console and click on your instance.
2. Look at the details panel at the bottom and copy the **Public IPv4 address**.



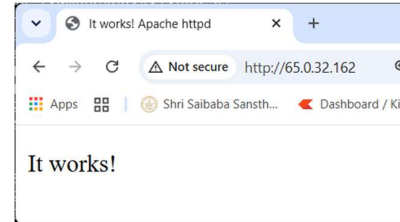


Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



3. Open a new tab in your web browser, paste the IP address into the URL bar (make sure to use http:// and not https://), and press Enter.
4. You should see a webpage displaying: **"It works!"**



Ideal Steps of Bash Script:

During configuration EC2 Instance you can define this script in User Data in an Advance. User Data normally runs automatically only during the instance's first boot.

- `#!/bin/bash` → Tells Linux to execute the script using Bash
- `yum update -y` → Updates installed packages
- `yum install -y httpd` → Installs Apache
- `systemctl start httpd` → Starts Apache immediately
- `systemctl enable httpd` → Starts Apache automatically after reboot
- `echo "..." > /var/www/html/index.html` → Creates/replaces Apache's default HTML page

Apache's default page is normally located at: `/var/www/html/index.html`. If the EC2 instance is already running, you don't need to recreate it just to test the corrected script.

Change the HTML: SSH into your existing EC2 instance and run:

```
sudo nano /var/www/html/index.html
```

It will open nano editor, modify

```
<!DOCTYPE html>
<html>
<head>
  <title>AWS EC2 Demo</title>
</head>
<body>
  <h1>Welcome to Apache Web Server on Amazon Linux!</h1>
  <p>This HTML page is running from my EC2 instance.</p>
</body>
</html><!DOCTYPE html>
<html>
<head>
  <title>AWS EC2 Demo</title>
</head>
<body>
  <h1>Welcome to Apache Web Server on Amazon Linux!</h1>
  <p>This HTML page is running from my EC2 instance.</p>
</body>
</html>
```

Save:

Use **Ctrl + O** and then **Enter** followed by **Ctrl + X** to exit.

Then refresh: <http://YOUR-PUBLIC-IP> example: <http://65.0.32.162/>



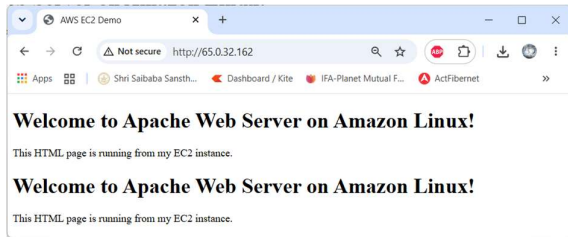


Cloud Computing & Dev-Ops

Transform Code into Cloud Solutions: Master AWS Today



You should see your HTML as shown below:



Step 4: Clean Up (Crucial to avoid charges)

To make sure you do not accidentally get billed when you are finished testing:

- Select your instance in the EC2 Dashboard.
- Click Instance state at the top right.
- Click Terminate instance. This completely

deletes the virtual machine and its attached storage so it stops consuming your free tier limits.

Connecting to your Linux EC2 instance from a Windows computer

Easiest way is using the built-in **PowerShell** application and the .pem private key file you downloaded during the instance creation step.

1. Open the File Explorer and navigate to the folder containing your downloaded .pem key file (such as your Downloads folder).
2. Open PowerShell in that specific folder by clicking into the address bar at the top of File Explorer, typing powershell, and pressing Enter.
3. Secure your private key file so the Windows SSH client accepts it. Run the following two commands in PowerShell (replace my-key.pem with your actual file name):

powershell

icacls my-key.pem /inheritance:r

icacls my-key.pem /grant:r "\$(\$env:USERNAME):(R)"

4. Retrieve your exact connection string from the Amazon EC2 Console by selecting your running instance, clicking **Connect**, and choosing the **SSH client** tab to copy the example command (which looks like `ssh -i "my-key.pem" ec2-user@your-public-ip`).
5. Paste that copied ssh command into your PowerShell window and press Enter. Type yes when prompted with a fingerprint authenticity warning to finish logging into your virtual server.
6. Install the test web server by running these standard Linux commands inside your PowerShell terminal:

bash

sudo yum update -y

sudo yum install -y httpd

sudo systemctl start httpd

sudo systemctl enable httpd

echo "<h1>Hello World from my AWS EC2 Instance via Windows!</h1>" | sudo tee /var/www/html/index.html

7. Verify it works by copying your instance's **Public IPv4 address** from the EC2 Dashboard, pasting `http://your-public-ip` into your Windows web browser, and checking for your greeting message.

🌐🔗🏠 **Ignite Innovation, Scale Success: Master AWS.** 🏠🌐🔗



Presented by
Ranjan Bhatnagar